

CEG 4350/5350 – Operating Systems Internals and Design

Project 4 – File Systems

Learning Objectives

- Learn how to implement a basic file system in your OS.

Overview

In this project you will implement a working version of the FAT file system with some simplifications and constraints. The FAT file system is one of the simplest file systems that is still in use today. After implementing this file system, your OS will be capable of creating files, opening files, reading and writing files, and deleting files that exist on your disk.

Even though the FAT file system is by comparison, simple, its implementation requires an in-depth understanding of the format of the FAT file system and its data structures. Anything more than basic file operations can be very complex to implement. For this reason we will focus only on a few basic file operations. A more advanced OS would be capable of copying files, creating links (shortcuts) between files, executing programs from disk, etc.

Floppy Disk Formatting

Since the first project, your floppy disk was formatted as a FAT16 file system. The structure is outlined below:

- bootloader.asm
 - Contains the boot sector
- fat.asm
 - Implements both copies of the file allocation tables
- root_dir.asm
 - Adds directory entries for root directory
 - Adds a directory entry that points to the kernel executable
- os.img
 - Combined 3 items in the following disk sectors:
 - Bootloader Sector 0 (0x0000 - 0x01FF)
 - FAT copies Sectors 1 – 22 (0x0200 - 0x2DFF)
 - Root directory Sector 23 (0x2E00 - 0x2FFF)
 - Kernel executable Sectors 24 – 78 (0x3000 - 0x9BFF)
 - NOTE: The FAT entries start at 2 which corresponds to physical sector 24
 - Cluster to sector conversion: $\text{sector} = \text{cluster} + 24 - 2$
 - Binary files can be easily read by adding a Notepad++ plugin called “HexEditor” or by downloading a free hex editor.

FAT File Operations

To implement basic file operations in our OS make the following modifications:

1. Modify project4.c:
 - a. `int writeFATs()`
 - i. Loops through both FAT tables ensuring each entry is consistent between both copies
 1. If the FAT tables are inconsistent, print an error message, return non-zero value
 - ii. Write both copies of the FAT tables to the disk
 1. Sectors for the first FAT table are 1 to 11
 2. Sectors for the second FAT table are 12 to 22
 3. If writing the FAT tables fails, print an error message, return non-zero value
 - iii. Return 0 if no error has occurred
 - b. `int writeCurrentDirectory()`
 - i. Checks if the `currentDirectory` is opened
 1. If `currentDirectory` is not opened, print an error message, return non-zero value
 - ii. Checks if the `currentDirectory` is the root directory, then writes the directory to the disk
 1. The root directory exists in sector 23
 2. If not the root directory, then loop through the FAT tables starting at `currentDirectory.directoryEntry->startingCluster` writing at each cluster
 3. If any write operations fail, print an error message, return non-zero value
 - iii. Return 0 if no error has occurred
 - c. `int writeCurrentFile()`
 - i. Checks if the `currentFile` is opened
 1. If `currentFile` is not opened, print an error message, return non-zero value
 - ii. Writes the file to the disk
 1. Loop through the FAT tables starting at `currentFile.directoryEntry->startingCluster` writing at each cluster
 2. If any write operations fail, print an error message, return non-zero value
 - iii. Return 0 if no error has occurred
 - d. `int resizeFile(uint16 startingCluster, uint32 newFileSize)`
 - i. Check if the number of clusters needed from the `newFileSize` is between 1 and `FAT_ENTRIES_MAX`, if not, print an error message, return non-zero value
 - ii. Loop through the FAT table starting from `startingCluster` until `0xFFFF` to check if the new cluster count is less than, greater than, or equal to the amount of clusters currently assigned
 - iii. If count is equal to currently assigned cluster count: return 0
 - iv. If count less than currently assigned cluster count:
 1. Loop through the FAT table from `startingCluster` and:
 - a. Set the "new" last cluster in the FAT table to `0xFFFF`
 - b. Overwrite the remaining clusters in the FAT table that were previously assigned to `0x0000`
 - v. If count greater than currently assigned cluster count:
 1. Loop through the FAT table and verify there are enough `0x0000` entries
 - a. If not, print an error message, return non-zero value
 2. Loop through the FAT table from `startingCluster` and:
 - a. Set the last cluster in the FAT table to point to the next free cluster found from `nextFreeCluster()`
 - b. Continue until the number of assigned clusters is equal to count
 - vi. Call `writeFATs()` to write the changes to the FAT tables back to the disk

The disk before creating any new files:

00000200	00 00 00 00 03 00 04 00 05 00 06 00 07 00 08 00	
00000210	09 00 0A 00 0B 00 0C 00 0D 00 0E 00 0F 00 10 00	
00000220	11 00 12 00 13 00 14 00 15 00 16 00 17 00 18 00	
00000230	19 00 1A 00 1B 00 1C 00 1D 00 1E 00 1F 00 20 00	
00000240	21 00 22 00 23 00 24 00 25 00 26 00 27 00 28 00	! . " . # . \$. % . & . ' . (.
00000250	29 00 2A 00 2B 00 2C 00 2D 00 2E 00 2F 00 30 00	) . * . + . , . - . . / . 0 .
00000260	31 00 32 00 33 00 34 00 35 00 36 00 FF FF 00 00	1 . 2 . 3 . 4 . 5 . 6
00000270	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00000280	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

First FAT Copy

00002E00	2E 20 20 20 20 20 20 20 20 20 20 10 00 00 00 00	
00002E10	00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00	
00002E20	2E 2E 20 20 20 20 20 20 20 20 20 10 00 00 00 00	
00002E30	00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00	
00002E40	6B 65 72 6E 65 6C 20 20 62 69 6E 00 00 00 00 00	k e r n e l . b i n
00002E50	00 00 00 00 00 00 00 00 00 00 00 02 00 00 C8 00 00	
00002E60	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Root Directory

The disk after creating 3 new files:

00000200	00 00 00 00 03 00 04 00 05 00 06 00 07 00 08 00	
00000210	09 00 0A 00 0B 00 0C 00 0D 00 0E 00 0F 00 10 00	
00000220	11 00 12 00 13 00 14 00 15 00 16 00 17 00 18 00	
00000230	19 00 1A 00 1B 00 1C 00 1D 00 1E 00 1F 00 20 00	
00000240	21 00 22 00 23 00 24 00 25 00 26 00 27 00 28 00	! . " . # . \$. % . & . ' . (.
00000250	29 00 2A 00 2B 00 2C 00 2D 00 2E 00 2F 00 30 00	) . * . + . , . - . . / . 0 .
00000260	31 00 32 00 33 00 34 00 35 00 36 00 FF FF FF FF	1 . 2 . 3 . 4 . 5 . 6
00000270	FF FF FF FF 00 00 00 00 00 00 00 00 00 00 00	

First FAT Copy

00002E00	2E 20 20 20 20 20 20 20 20 20 20 10 00 00 00 00	
00002E10	00 00 00 00 00 00 00 00 00 00 00 00 02 00 00	
00002E20	2E 2E 20 20 20 20 20 20 20 20 20 10 00 00 00 00	
00002E30	00 00 00 00 00 00 00 00 00 00 00 00 02 00 00	
00002E40	6B 65 72 6E 65 6C 20 20 62 69 6E 00 00 00 00 00	k e r n e l . b i n
00002E50	00 00 00 00 00 00 00 00 00 00 02 00 00 C8 00 00	
00002E60	66 69 6C 65 61 20 20 20 74 78 74 00 00 00 00 00	f i l e a . t x t
00002E70	00 00 00 00 00 00 00 00 00 00 37 00 00 02 00 00	 7
00002E80	66 69 6C 65 62 20 20 20 74 78 74 00 00 00 00 00	f i l e b . t x t
00002E90	00 00 00 00 00 00 00 00 00 00 38 00 00 02 00 00	 8
00002EA0	66 69 6C 65 63 20 20 20 74 78 74 00 00 00 00 00	f i l e c . t x t
00002EB0	00 00 00 00 00 00 00 00 00 00 39 00 00 02 00 00	 9

Root Directory

00009A00	74 68 69 73 20 69 73 20 69 6E 20 66 69 6C 65 20	t h i s . i s . i n . f i l e
00009A10	61 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	a
00009A20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

file a.txt

00009C00	74 68 69 73 20 69 73 20 69 6E 20 66 69 6C 65 20	t h i s . i s . i n . f i l e
00009C10	62 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	b
00009C20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

file b.txt

00009E00	74 68 69 73 20 69 73 20 69 6E 20 66 69 6C 65 20	t h i s . i s . i n . f i l e
00009E10	63 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	c
00009E20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

file c.txt

The disk after expanding file b.txt by an additional sector (while leaving file a.txt and file c.txt unmodified):

```
00000200 00 00 00 00 03 00 04 00 05 00 06 00 07 00 08 00 . . . . .
00000210 09 00 0A 00 0B 00 0C 00 0D 00 0E 00 0F 00 10 00 . . . . .
00000220 11 00 12 00 13 00 14 00 15 00 16 00 17 00 18 00 . . . . .
00000230 19 00 1A 00 1B 00 1C 00 1D 00 1E 00 1F 00 20 00 . . . . .
00000240 21 00 22 00 23 00 24 00 25 00 26 00 27 00 28 00 ! . " . # . $ . % . & . ' . ( .
00000250 29 00 2A 00 2B 00 2C 00 2D 00 2E 00 2F 00 30 00 ) . * . + . , . - . . / . 0 .
00000260 31 00 32 00 33 00 34 00 35 00 36 00 FF FF FF FF 1 . 2 . 3 . 4 . 5 . 6 . . . .
00000270 3A 00 FF FF FF FF 00 00 00 00 00 00 00 00 00 00 : . . . . .
00000280 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000290 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
```

First FAT Copy

```
00002E00 2E 20 20 20 20 20 20 20 20 20 10 00 00 00 00 . . . . .
00002E10 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00 . . . . .
00002E20 2E 2E 20 20 20 20 20 20 20 20 10 00 00 00 00 . . . . .
00002E30 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00 . . . . .
00002E40 6B 65 72 6E 65 6C 20 20 62 69 6E 00 00 00 00 00 k e r n e l   b i n . . . .
00002E50 00 00 00 00 00 00 00 00 00 00 02 00 00 C8 00 00 . . . . .
00002E60 66 69 6C 65 61 20 20 20 74 78 74 00 00 00 00 00 f i l e a       t x t . . . .
00002E70 00 00 00 00 00 00 00 00 00 00 37 00 00 02 00 00 . . . . . 7 . . . .
00002E80 66 69 6C 65 62 20 20 20 74 78 74 00 00 00 00 00 f i l e b       t x t . . . .
00002E90 00 00 00 00 00 00 00 00 00 00 38 00 00 04 00 00 . . . . . 8 . . . .
00002EA0 66 69 6C 65 63 20 20 20 74 78 74 00 00 00 00 00 f i l e c       t x t . . . .
00002EB0 00 00 00 00 00 00 00 00 00 00 39 00 00 02 00 00 . . . . . 9 . . . .
00002EC0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
```

Root Directory

```
00009C00 74 68 69 73 20 69 73 20 69 6E 20 66 69 6C 65 20 t h i s   i s   i n   f i l e
00009C10 62 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 b . . . . .
00009C20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00009C30 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
```

file b.txt (first sector)

```
0000A000 74 68 69 73 20 69 73 20 69 6E 20 74 68 65 20 73 t h i s   i s   i n   t h e   s
0000A010 65 63 6F 6E 64 20 73 65 63 74 6F 72 20 6F 66 20 e c o n d   s e c t o r   o f
0000A020 66 69 6C 65 20 62 00 00 00 00 00 00 00 00 00 00 f i l e   b . . . . .
0000A030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
0000A040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
```

file b.txt (second sector)

How to Submit

To submit your code, commit your changes to your GitHub Classroom repository and include a link to your repository in the Pilot dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
writeFATs() is properly implemented	1.00
writeCurrentDirectory() is properly implemented	1.00
writeCurrentFile() is properly implemented	1.00
resizeFile() can grow allocated clusters	1.00
resizeFile() can shrink allocated clusters	1.00
Total	5.00